Rapport de Projet de PFA

Lucas Rooy, Sacha Le Hir MazŽ

PrŽsentation du Jeu et tutoriel

Il sÔagit dÔun jeu de puzzle, basŽ sur des chapeaux que lÔon peut prendre en main et lancer, en pouvant se servir de leurs effets.

Il existe trois chapeaux diffŽrents dans le jeu :

¥ Le fez lorsque lancŽ contre une surface

¥ Le haut-de-forme que lÔon peut utiliser comme trampoline

¥ Le bŽret qui se comporte comme une plateforme volante

Le but du jeu est de rejoindre un porte-manteau en utilisant les chapeaux. Le porte-manteau constitue alors une porte permettant de passer au niveau suivant, ou de finir le jeu.

Sur le build `SDL`, le jeu utiliser les touches suivantes :

¥ `q` pour se dŽplacer ˆ gauche

¥ `d` pour se dŽplacer ˆ droite

¥ `[SPACE]` pour sauter

¥ `e` pour interagir

  ! LorsquÔon porte un chapeau, cela permet de le lancer

  ! LorsquÔon est ˆ proximitŽ dÔun chapeau, cela permet de le porter

Pour le lancer, il suffit de faire `make run_sdl`, car nous nous sommes concentrŽs sur la version SDL du jeu. Il est possible dÔutiliser Nix pour obtenir un environnement de dŽveloppement, en faisant `nix develop` ˆ la racine.

Nous possŽdons actuellement quatre niveaux.

¥ Un premier niveau permet simplement de se familiariser avec les contr™les

¥ Un second niveau permet de dŽcouvrir le fonctionnement du haut de forme

¥ Un troisi•me niveau permet de dŽcouvrir le fonctionnement du fez

¥ Un quatri•me niveau permet de dŽcouvrir le fonctionnement du bŽret

¥ Un cinqui•me niveau montre les possibilitŽs de puzzle du jeu

¥ Un sixi•me niveau pour dire au revoir!

Le fez

Le fez explose les blocs ˆ une distance maximale de `30` pixels autour de lui lorsque projetŽ contre une surface ˆ une vitesse de norme minimale `0.4`, qui est grossi•rement la vitesse de lancer par dŽfaut.

Le bŽret

CÒest la seule entitŽ (en dehors des blocs) immune ˆ la gravitŽ et aux frottements du sol. Ainsi, une fois lancŽ (ou poussŽ), il devient une plateforme mouvante permettant de traverser des puits.

Le haut de forme

Le haut de forme se comporte comme un trampoline : sauter dessus augmente la vitesse du saut de 0.5 pixels/seconde ˆ 1.2 pixels/seconde et permet dÒaccŽder ˆ des zones inaccessibles.lateforme permettant de traverser des puits.

Le porte-chapeau

Il sÒagit de lÒobjectif du jeu ! A lÒaide des chapeaux, il faut rejoindre cet objet prŽsent sur la carte et appuyer sur la touche e ˆ proximitŽ du porte-chapeau afin de passer au niveau suivant.

PrŽsentation du Code

PrŽambule : Modification de Gfx et Ecs

La boucle est appelŽe avec un couple `(ticks, dt)` (le nombre de ticks (unitŽ pas importante) depuis le dŽbut du jeu, le nombre de ticks depuis la derni•re actualisation). C'est plus pratique.

Sur suggestion de Adrien, et avec son patch, nous avons modifiŽ `Ecs` qui ne supprimait pas correctement les entitŽs.

Composants et entitŽs

Nous avons dŽfinis des composants permettant de crŽer les types d'entitŽs :

- `drawable`, pouvant •tre dessinŽ
- `animated`, pouvant •tre dessinŽ et animŽ. Dans ce cas, le champ `tick_speed` correspond au nombre de ticks ˆ attendre avant de changer de texture. Le champ `animation_callback` est un thunk, appelŽ lorsque l'animation a effectuŽ un tour (et est sur le point de revenir ˆ 0).
- `collidable`, pouvant •tre sujet aux collisions
- `physics`, pouvant •tre dŽplacŽ et accŽlŽrŽ par les syst•mes `Move` et `Physics`.
- `explodable`, pouvant •tre explosŽ. Cet entity type est aussi utilisŽ par l'entitŽ `explosion`, qui porte alors un tag particulier indiquant la force de l'explosion.

Voici un inventaire des entitŽs que nous avons ajoutŽes :

- `player` est un joueur
- `hat` est une entitŽ reprŽsentant un chapeau, qu'il soit portŽ ou non
  - Si le chapeau est portŽ, alors il est dŽsenregistrŽ de tous les syst•mes et ajoutŽ dans le tag du joueur (tag `Player of hat`)
  - Si le chapeau est prŽsent dans le monde, alors il est enregistrŽ dans les syst•mes appropriŽs et prŽsent dans la liste `wild_hats` accessible via `Global`.
- `block` est un rectangle ayant une texture, une position, une taille, et la possibilitŽ d'•tre cassŽ (par une explosion). Il s'agit d'une entitŽ tr•s prŽsente dans le jeu car elle est utilisŽe pour construire les parois du puzzle.
- `explosion` est une explosion. Par simplicitŽ, elle est de sur-type `explodable`. Le centre de l'explosion et le centre de sa box (ajoutŽ ˆ sa position), et la force de l'explosion est alors la valeur de `force` dans son tag qui doit •tre `TExplosion force`. Si le tag est diffŽrent, l'explosion est considŽrŽ comme un `explodable` classique (pouvant •tre explosŽ).

Syst•mes

Les syst•mes `Physics`, `Move` et `Collision` sont exactement ceux faits durants les TPs.

Nous possŽdons aussi un syst•me `Animation`, qui se charge de modifier la texture d'un objet lorsque sa `tick_speed` est passŽe, afin de l'animer.

Le syst•me `Camera` est intŽressant, car il permet de dŽfinir une camŽra ayant une position et un zoom; cela est utile pour lÔeffet de tremblement lors dÔune explosion

Enfin, le syst•me `Explosion_sys` (nommŽ de cette fa•on pour ne pas rentrer en collision avec le module de lÔentitŽ `Explosion`) rŽgit les explosions.

Architecture gŽnŽrale

Le code est structurŽ autour du module `Level`, qui permet de charger et dŽcharger des niveaux, du module `Game`, qui constitue lÔentrypoint du jeu et se charge de lÔinitialisation et du chargement des textures, et du module `Inputs`, qui permet lÔinteraction avec le joueur.

Architecture des chapeaux

Les chapeaux ont des fonctionnalitŽs complexes, diffŽrentes pour chacun dÔentre eux :

Le Fez

On utilise le resolver des collisions pour dŽtecter lorsquÔil faut faire une explosion. D•s que cÔest le cas, on supprime le fez et on crŽe une entitŽ explosion ˆ sa position.

Le Haut de Forme

Celui-ci poss•de son fonctionnement dans le resolver des collisions du player, car cÔest dans ce resolver que lÔon dŽcide le type de surface sur laquelle le player se trouve.

Si cÔest un Haut de Forme, alors on saura au moment de sauter quÔil faut sauter avec une vitesse plus ŽlevŽe

Le BŽret

Celui-ci est plus complexe. Dans son resolver, on remet sa hauteur et sa vitesse aux bonnes v

Architecture des chapeaux

En gŽnŽral, les chapeaux peuvent •tre attrapŽs (nearest neighbour) par le joueur et portŽs, apr•s quoi on dŽsactive tous leurs syst•mes, et leur affichage et rŽalisŽ en m•me temps que celui du joueur.

LorsquÔil est lancŽ, le comportement peut •tre un peu diffŽrent entre certains chapeaux, mais on le rŽenregistre simplement dans leur syst•mes.

Les chapeaux ont des fonctionnalitŽs complexes, diffŽrentes pour chacun dÔentre eux :

Le Fez

On utilise le resolver des collisions pour dŽtecter lorsquÔil faut faire une explosion. D•s que cÔest le cas, on supprime le fez et on crŽe une entitŽ explosion ˆ sa position.

Le Haut de Forme

Celui-ci poss•de son fonctionnement dans le resolver des collisions du player, car cȇest dans ce resolver que lȇon dŽcide le type de surface sur laquelle le player se trouve.

Si cȇest un Haut de Forme, alors on saura au moment de sauter quȇil faut sauter avec une vitesse plus ŽlevŽe

Le BŽret

Celui-ci est plus complexe. Dans son resolver, on remet sa hauteur et sa vitesse aux bonnes valeurs, et dans le syst•me des collisions, on met lȇelasticitŽ entre le BŽret et les murs ˆ 1, afin quȇil ne soit pas ralenti de toute fa•on (inutile apr•s coup, il sȇagit dȇun artefact).

PrŽsentation de deux points techniques

Interpolation du temps pour les collisions

Pour faire la gestion des collisions, on utilisait au dŽpart ce code simpliste :

game.ml (avant)

```
let update (ticks, dt) =
Ê   let _ = Input.handle_input (ticks, dt) in
Ê   Physics_system.update (ticks, dt);
Ê   Move_system.update (ticks, dt);
Ê   Collision_system.update (ticks, dt);
Ê   Camera_system.update (ticks, dt);
Ê   Animation_system.update (ticks, dt);;
```

Or, si pour une raison ou une autre, le `dt` entre deux appels de `update` est trop grand (cÔest-ˆ-dire, suffisamment grand pour quÔun objet avec une vitesse suffisamment ŽlevŽe traverse un obstacle sans dŽclancher de collisions), certaines collisions sont ignorŽes.

Cette situation se produit dans beaucoup de cas :

- Si, peut-•tre, le syst•me dÔexploitation dŽcide de laisser dormir notre jeu pendant trop longtemps. Dans ce cas, on pourrait utiliser `nice` pour sans sortir, ˆ la rigueur, mais ce nÔest pas optimal
- Si les calculs et lÔaffichage de la frame prŽcŽdente ont ŽtŽ trop longs; cela se produit couramment, en particulier lorsque nous avons beaucoup dÔentitŽs prŽsentes.

Afin de rŽgler le probl•me, on effectue une interpolation du temps ˆ chaque update.

Supposons quÔentre deux appels ˆ `update` se soient passŽes `dt=200ms`. On effectuera alors les Žtapes de calcul (move, physics, etc) 200 fois, avec un pas de temps de `1ms` ˆ chaque fois, afin de ne pas perdre en prŽcision.

Ainsi, chaque grand pas de temps est dŽcomposŽ en petits pas de temps (donc en plus petits mouvements) que lÔon peut analyser finement; on a moins de chances de perdre des collisions!

Bien sžr, cette approche poss•de des inconvŽnients, dont un particuli•rement visible : son impact sur les performances non nŽgligeable, pour y pallier, nous divisons par `5` le nombre de pas.

game.ml (apr•s)

```
let last_ticks = ref 0.

let update (ticks, dt) =
Ê if !last_ticks +. 2000. < ticks
Ê then (
Ê   Printf.printf "Current dt = %f\n" dt;
Ê   last_ticks := ticks);
```

```
Ê for i = 0 to int_of_float (ceil (dt /. 5.)) do
Ê   let dt = ticks, dt /. ceil (dt /. 5.) in
Ê   let () = Input.handle_input dt in
Ê   Physics_system.update dt;
Ê   Move_system.update dt;
Ê   Collision_system.update dt
Ê done;
Ê Camera_system.update (ticks, dt);
Ê Animation_system.update (ticks, dt);
Ê None
;;
```

XÕYÕZ'

Pour stocker les niveaux, on a dŽcidŽ dÕutiliser les types suivants :

```
type layer =
Ê { contents : string;
Ê   offsets : (selector, Vector.t ) Hashtbl.t;
Ê   width : int;
Ê   stride : Rect.t;
Ê }

type level =
Ê { layers : layer list;
Ê   camera : float * Vector.t;
Ê }
```

Les niveaux sont des quadrillages de blocks de taille ŽlŽmentaire donnŽe par stride avec width qui dicte le nombre de blocks par ligne. Le contenu de chaque layer est sŽrialisŽ sous forme de chaine de caract•res o• chaque caract•re reprŽsente un certain objet. La champ offset permet plus de prŽcision si besoin.

Dans notre cas, on a : *x : block *f : fez *h : haut de forme *b : bŽret *p : porte chapeau

```
let f chr layer position =
Ê if chr = 'x' then
Ê   Block.create layer position Rect.{ width = 20; height = 20 } Texture.black |>
ignore
Ê else if chr = '@' then
Ê   Player.create layer position [| Global.get_texture "extra_character_a" |] |>
ignore
Ê else if chr = 'f' then
Ê   Hat.create position.x position.y layer (Global.get_texture "fez") Fez
Ê else if chr = 'h' then
Ê   Hat.create position.x position.y layer (Global.get_texture "hdf") Hdf
Ê else if chr = 'b' then
Ê   Hat.create position.x position.y layer (Global.get_texture "beret") (Beret
```

```
position.y)
Ê else if chr = 'p' then
Ê   Pc.create position.x position.y layer (Global.get_texture "pc")
Ê else ()
```

Un niveau est donc donnŽ par une liste de layers ce qui permet dÔavoir une superposition de calque.

Pour load un niveau, on utiise la fonction suivante :

```
Ê let load f lvl =
Ê   List.iteri
Ê       (fun layer_idx layer ->
Ê       String.iteri
Ê           (fun idx chr ->
Ê               let x = idx mod layer.width * layer.stride.width in
Ê               let y = idx / layer.width * layer.stride.height in
Ê               let position = Vector.add Vector.{ x = float x; y = float y }
(get_offset x y layer chr) in
Ê               f chr layer_idx position)
Ê           layer.contents)
Ê       lvl.layers;
Ê   let zoom, pos = lvl.camera in
Ê   Global.update (fun g -> { g with camera = { zoom; pos } })
```

La gestion des layers est ainsi naturellement gŽrŽe par lÔordre par laquelle on it•re sur la variable layer.

MŽthodologie de tests

Nous avons plusieurs fonctionnalitŽs ˆ tester :

- ¥ Les chapeaux
- ¥ La physique
- ¥ Les niveaux

Voici notre mŽthode :

FaisabilitŽ des niveaux

Tout dÕabord, il faut nous assurer que nos niveaux sont finissables, donc nous les testons en jouant afin de voir si le niveau se termine.

LÕidŽe est aussi de noter les probl•mes rencontrŽs : de physique, de fonctionnement du chapeau, ou de gŽomŽtrie du niveau.

Si le probl•me concerne uniquement le niveau, il est possible de le modifier pour le rendre finissable.

Les niveaux sont ce que nous avons de plus proche dÕune suite de tests reproductibles, il sÕagit donc de la base des tests et de la dŽcouverte des bugs (apr•s tout, si un bug nÕest pas dŽclanchŽ dans les niveaux, est-ce vraiment un bug :) ?)

Chapeaux

Les chapeaux introduisent des comportements complexes au jeu quÕil est nŽcessaire de tester.

LorsquÕon voit quÕun chapeau se comporte Žtrangement, on commence par essayer dÕisoler le probl•me : test sur un niveau plat avec juste la gŽomŽtrie nŽcessaire pour le reproduire.

Physique

CÕest lˆ que cÕest particuli•rement amusant : nous avons essayŽ de marcher dans des murs, lancer des chapeaux dans des murs, sauter et marcher dans des murs, faire des double-sauts.

Certains chapeaux interagissent avec la physique; en particulier le bŽret. Pour ce dernier, il a fallu faire pas mal dÕessais (sauter sur le bŽret, bloquer le bŽret, lancer le bŽret contre un mur, dans le vide, etc) afin dÕidentifier les probl•mes.

Outils techniques

Afin de mener les tests ˆ bien, nous nous sommes dotŽs de quelques outils techniques.

DŽplacement arbitraire du joueur

Pour pouvoir tester plus rapidement, il est utile de pouvoir contr™ler les entitŽs (et en particulier le joueur) comme bon nous semble.

A cet effet, un clique nÛimporte o• sur la fen•tre permet de dŽclencher une tŽlŽportation du joueur au niveau du curseur.

Affichages terminal

Notre jeu ne demande pas ŽnormŽment de debuggage, ainsi, une solution satisfaisante pour le suivi des tests est dÛutiliser le temrinal pour afficher des informations sur la vitesse dÛune entitŽ, sa position, la dŽtection dÛune collision, etcÉ

Utilisation de la couleur

Pour correctement visualiser les collisions, nous avons utilisŽ au dŽbut de notre projet un affichage avec des couleurs diffŽrentes selon les collisions en cours.

RŽpartition du travail

Lucas

NOTE	Comme jÒŽcris du code tr•s vite, jÒai implŽmentŽ beaucoup du code du projet; nŽanmoins chaque fonctionnalitŽ, code et choix technique est le fruit dÒun travail de discussion dans le bin™me.

Ces parties du jeu me sont attribuables en grande partie:

- Le syst•me dÒaffichage avec zoom et positionnement de la camŽra
- Le syst•me de `Level`, permettant de charger/dŽcharger des niveaux du jeu
- Le syst•me permettant dÒattraper/lancer des chapeaux
- Le syst•me dÒinterpolation du temps pour permettre de ne pas rater les collisions.
- Le syst•me de build reproductibles avec Nix

NOTE	Apr•s beaucoup de discussions en bin™mes; jÒai tapŽ le code, mais il est le fruit dÒun travail commun.

Sacha

- Le syst•me de collisions
- Le porte chapeau
- Les textures
- Le map-making
- Le code de chargement des textures par continuations